



Sistemas Distribuidos I (75.74)

Interfaces y Protocolos

Layers, Tiers, Interfaces, Protocolos, Mensajes, Restful

Docentes

- | | | |
|---------------------|--------------------|-------------------|
| ■ Pablo D. Roca | ■ Nicolás Zulaica | ■ Máximo Gismondi |
| ■ Gabriel Robles | ■ Franco Papa | ■ Camila Sebellín |
| ■ Franco Barreneche | ■ Manuel Reberendo | |



● Modelos de Capas: Layers y Tiers

- Interfaces
- Protocolos
- Mensajes
- RESTful

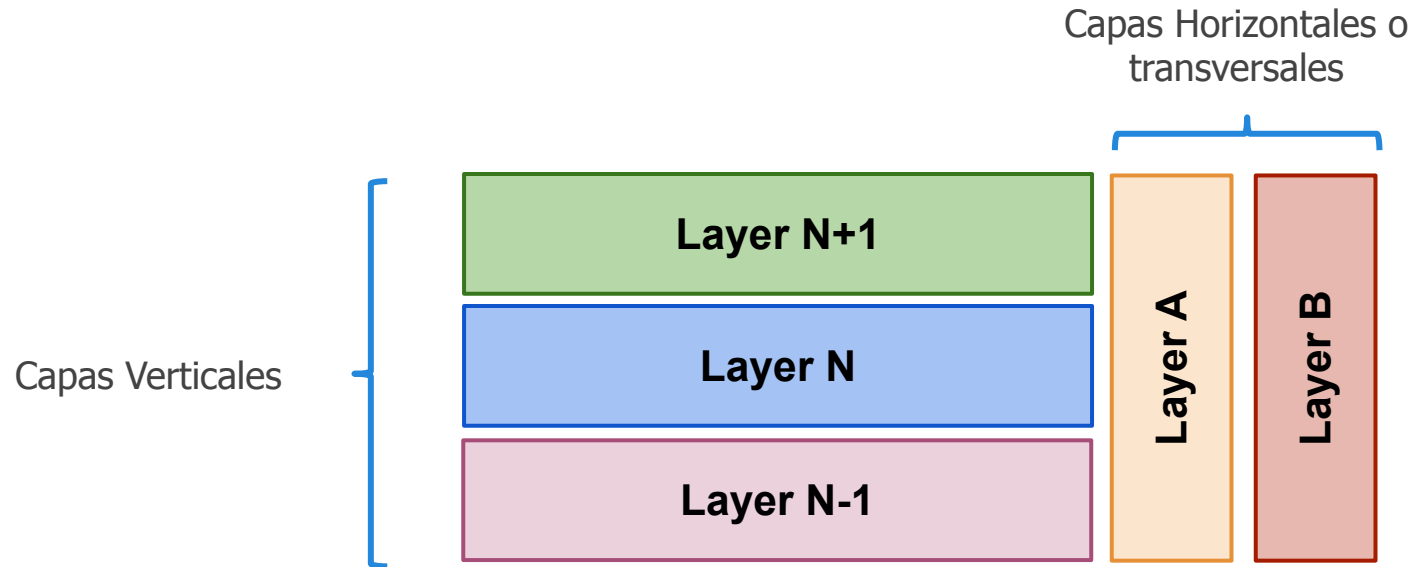


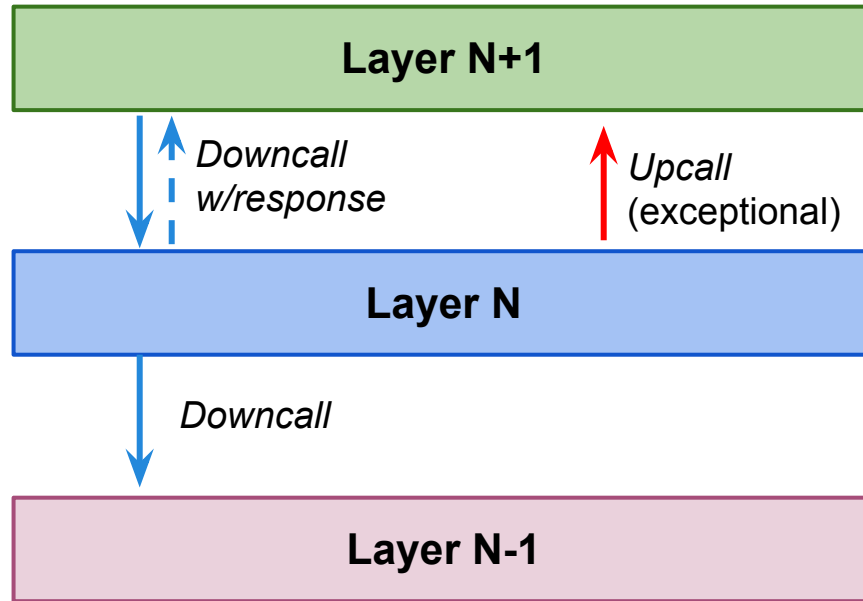
Arquitecturas de Capas

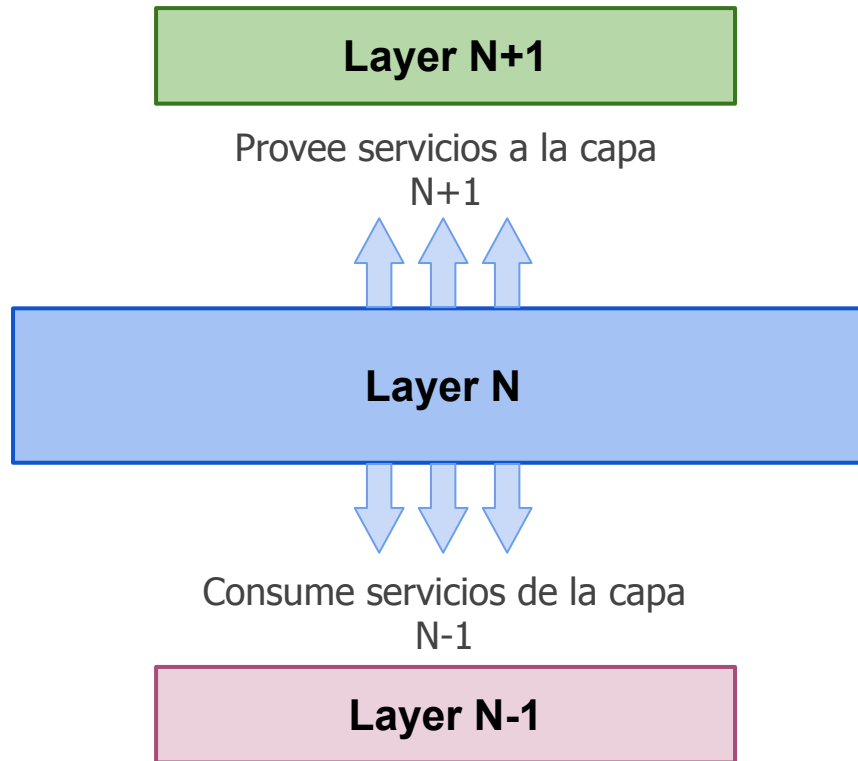
- Permiten dividir el problema en sub-problemas
- Fomentan el uso de interfaces
- Permiten intercambiar componentes reutilizando conectores y protocolos ya definidos
- Existen dos tipos de separación por capas:
 - *Layers*, o capas lógicas
 - *Tiers*, o capas físicas



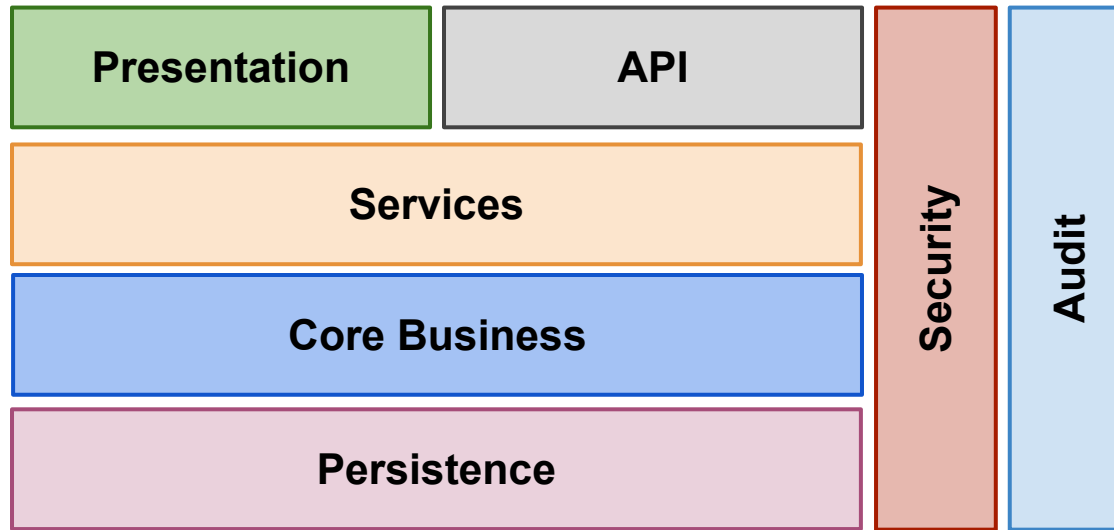
Agrupación lógica de componentes y funcionalidades de un sistema.





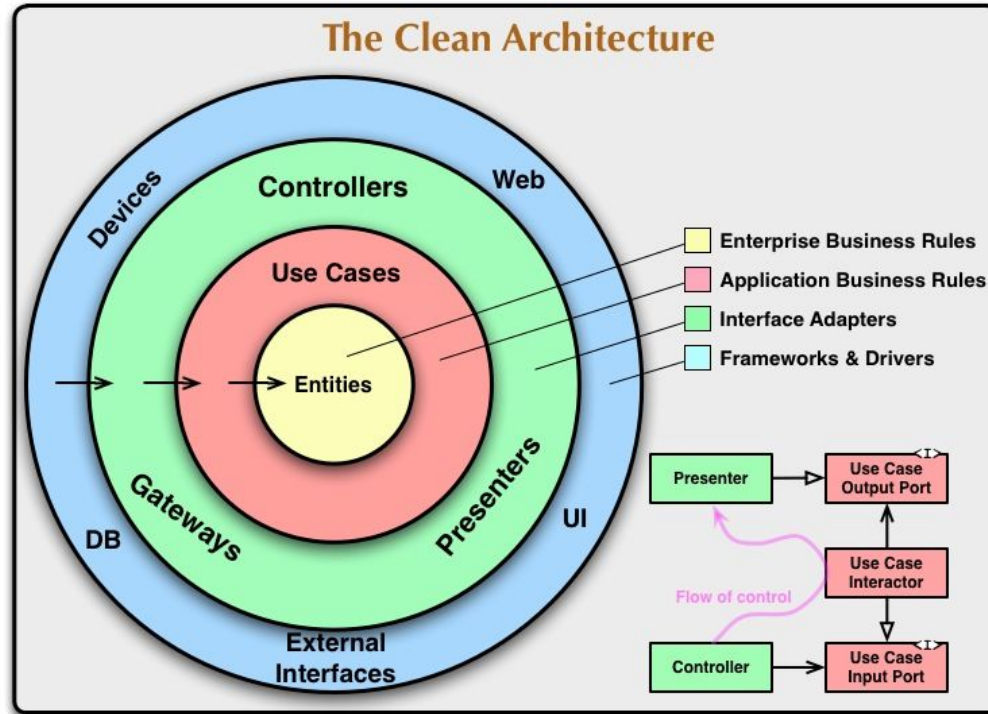


Módulo con responsabilidades limitadas, coherencia y cohesión



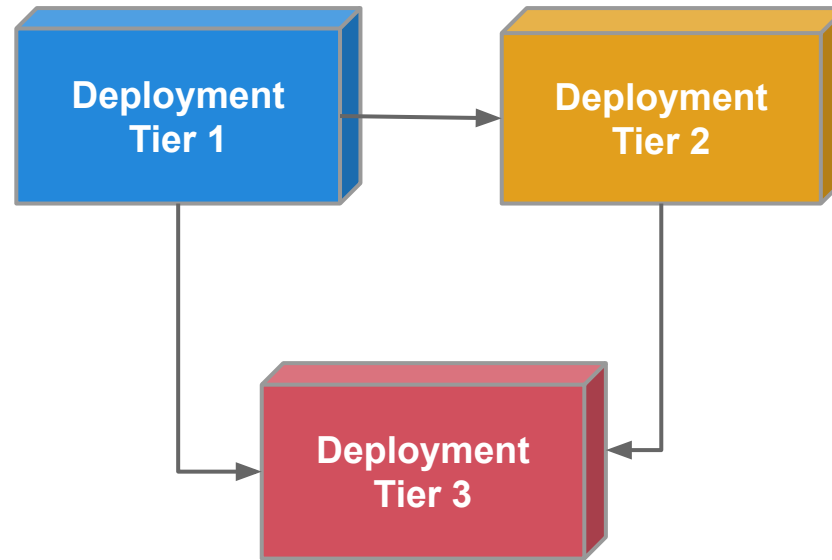


Layers | Ejemplo: Onion Architecture



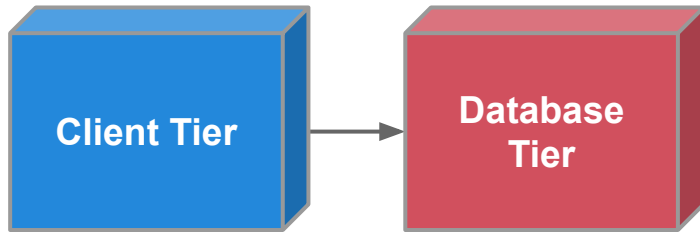


Describen la distribución física de componentes y funcionalidad de un sistema.

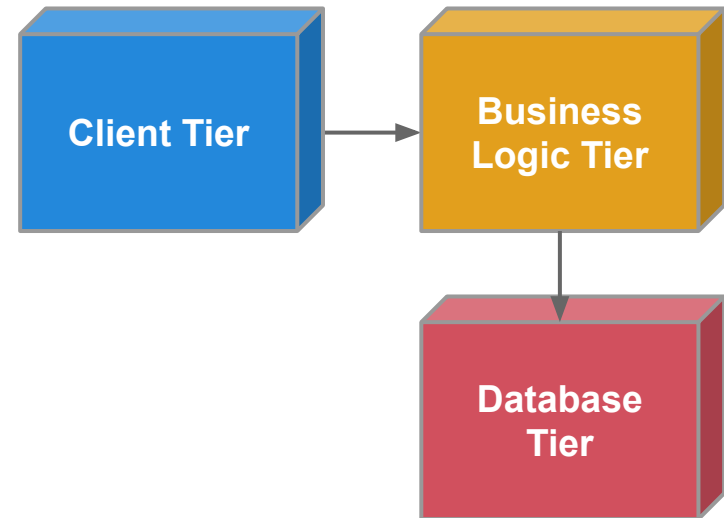


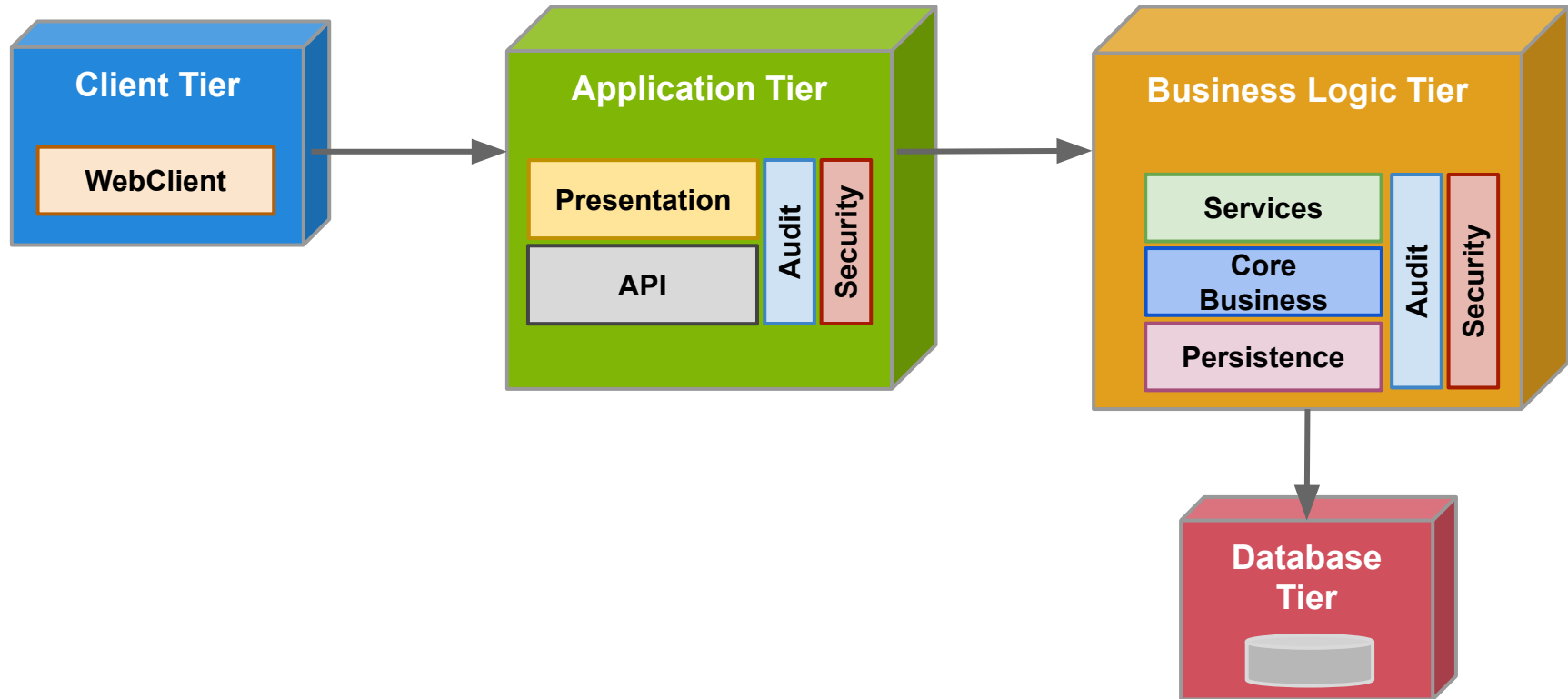


2-Tier Deployment



3-Tier Deployment





Agenda



- ☐ Modelos de Capas: Layers y Tiers
- ☒ **Interfaces**
- ☐ Protocolos
- ☐ Mensajes
- ☐ RESTful



Interfaces | Introducción

- Permiten la comunicación entre dos o más componentes/servicios/sistemas
- Diferentes contratos permiten diferentes clientes
- Sólo se expone **una parte** del sistema
- Esconden implementación
 - Implementación puede ser modificada sin cambiar contrato
 - Cambio de contrato implica una nueva versión



<https://xkcd.com/1770/>



Inter-Aplicaciones	Intra-Aplicaciones
Application Programming Interfaces (APIs) Ej: • Cliente por consola consultando Web Server • JS de navegador consultando servidor • Servicio consultando Servicio	Facades - Mediators - Interfaces Ej: • <i>Layer 2</i> consultando <i>Layer 1</i> • Mensaje enviado a un objeto local • Mensaje enviado a un objeto remoto (?)



Interfaces | Problemas a resolver

- **Software es difícil de cambiar**
 - ¿Qué pasa si un sistema está altamente acoplado a una API y ésta cambia?
 - ¿Cómo cambiamos una API ya definida?
- **Software es difícil de integrar**
 - No todos los componentes exponen interfaces útiles
 - Complejidad aumenta exponencialmente con la cantidad de componentes a integrar





Orientados a entidades

- Desacoplamiento entre sistemas.
- Flexibilidad como objetivo.
- Funcionalidad estándar.

Orientados a procesos

- Componentes altamente acoplados.
- Alta performance como objetivo.
- Funcionalidad diversa.

Wikipedia/Wikimedia REST API

```
GET /page/title/{title}
POST /page/html/{title}
GET /page/media/{title}
GET /feed/featured/{yyyy}/{mm}/{dd}
```

```
POST /transform/html/to/wikitext/{title}
POST /media/math/check/tex
GET api.php?action=query&
    titles=Doesntexist%7CMain&format=json
```




Web APIs

- Web Services based APIs ([SOAP](#))
- REST based APIs

Library-based / Frameworks

- Java API
 - Ej.: OpenJDK vs Oracle JDK
- Android API

Remote APIs

- Custom TCP/UDP services
- Object oriented: CORBA, JavaRMI
- Procedure oriented: RPC, gRPC

OS related

- POSIX
 - Ej.: Linux vs OpenBSD
- WinAPI

Agenda



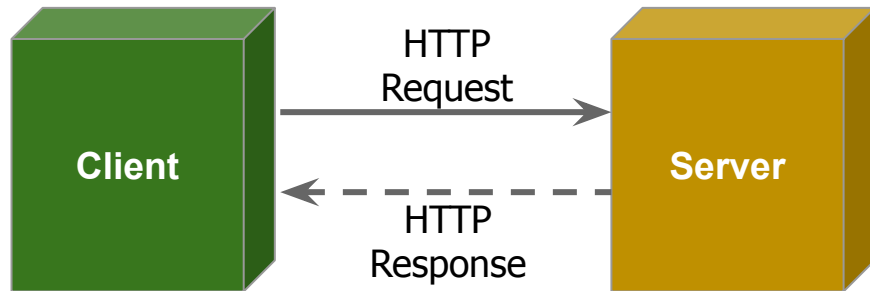
- ☐ Modelos de Capas: Layers y Tiers
- ☐ Interfaces
- ☒ **Protocolos**
- ☐ Mensajes
- ☐ RESTful



Protocolos | Modelo HTTP

Características:

- Modelo *Client-Server*
- Modelo *Request-Reply*
- Sin estado (servidor)



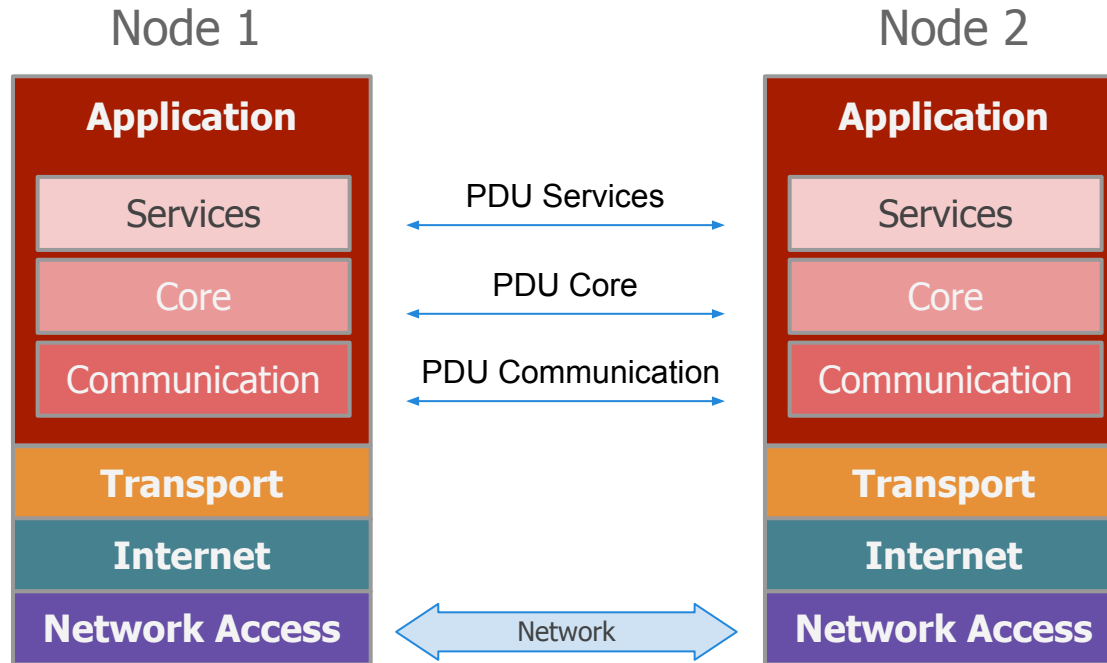
HTTP Request

```
GET https://en.wikipedia.org/wiki HTTP/1.1
Host: www.example.com
Accept: text/html,application/xhtml+xml;
Accept-encoding: gzip, deflate, br
Accept-language: en-US,en;q=0.9,es;q=0.8
```

HTTP Response

```
HTTP/1.1 200 OK
Content-encoding: gzip
Content-language: en
Content-length: 18558
Content-type: text/html; charset=UTF-8
```

```
<!DOCTYPE html>
<html class="client-nojs" lang="en" dir="ltr">
<head>
<meta charset="UTF-8"/>
...
</html>
```



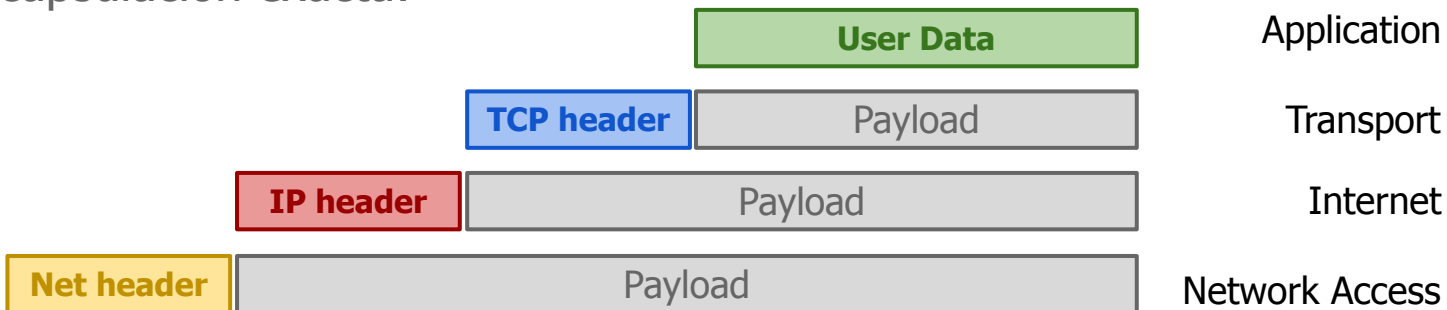


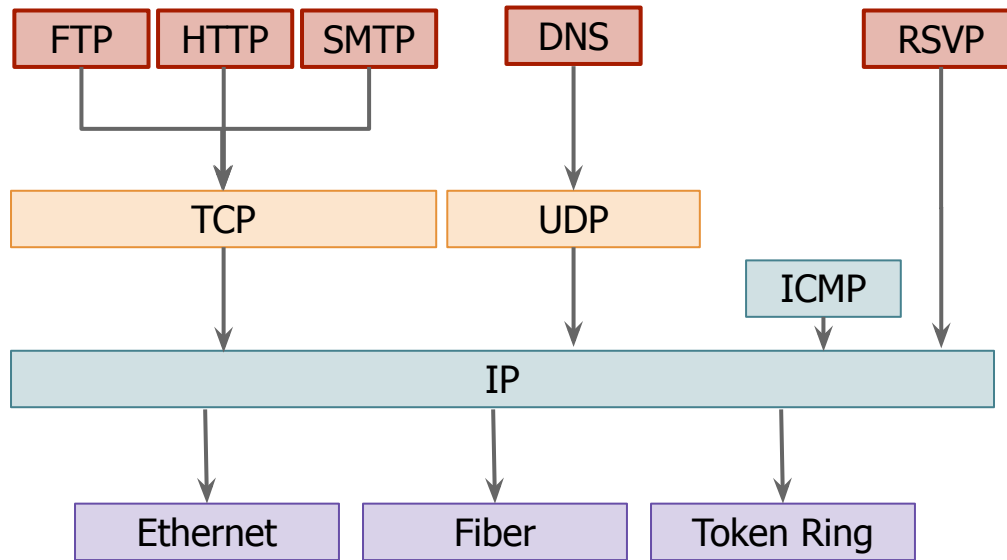
Protocolos | *Protocol Data Unit (PDUs)*

Encapsulación de PDUs entre capas:

1. Encapsulación exacta
2. Segmentación de paquetes
3. *Batching* de paquetes

Ej. Encapsulación exacta:





Application

Transport

Internet

Network Access

Agenda



- ☐ Modelos de Capas: Layers y Tiers
- ☐ Interfaces
- ☐ Protocolos
- ☒ **Mensajes**
- ☐ RESTful



Mensajes | Formato de Paquetes | Binario

- **Alta performance**
 - Tamaño de mensajes eficientes
 - Compresión puede no ser necesaria
- **Serialización**
 - Autogeneración de código
(e.g. Google Protobuf, Thrift, ASN.1)
 - No siempre existe soporte en todos los lenguajes
- **Interacción**
 - Cliente específico para cada aplicación
 - Decoder para interpretar los mensajes



Mensajes | Formato de Paquetes | Ejemplo (Protobuf)

person.proto

```
message Person {  
    required string name = 1;  
    required int32 id = 2;  
    optional string email = 3;  
}
```

Generación de Código

```
protoc -I=$SRC_DIR --cpp_out=$DST_DIR  
$SRC_DIR/person.proto
```

Serialización (Java)

```
Person john = Person.newBuilder()  
    .setId(1234)  
    .setName("John Doe")  
    .setEmail("jdoe@example.com")  
    .build();  
output = new  
FileOutputStream(args[0]);  
john.writeTo(output);
```

Deserialización (C++)

```
Person john;  
fstream input(argv[1],  
    ios::in | ios::binary);  
john.ParseFromIstream(&input);  
id = john.id();  
name = john.name();  
email = john.email();
```



Mensajes | Formato de Paquetes | Texto plano

- **Baja performance**
 - *Throughput* bajo
 - Compresión agrega *overhead*
- **Serialización**
 - Formatos *human-readable* (JSON, XML)
 - Serialización básica (e.g. HTTP, SMTP)
- **Interacción**
 - Cliente único si se conoce el protocolo (e.g. cURL + REST API)
 - Fácil de *debuggear*



Mensajes | Formato de Paquetes | Ejemplo (CURL)

```
user@hostname:~$ curl -H "Content-Type: application/json" \  
-X POST \  
-d '{"username":"lalala","password":"supersecurepass"}' \  
http://localhost:8080/api/login/
```

Request

```
POST /login HTTP/1.1  
Host: localhost:8080  
User-Agent: curl/7.58.0  
Accept: */*  
Content-Type: application/json  
Content-Length: 50  
  
{"username":"lalala","password":"supersecurepass"}
```



Mensajes | Longitud de Paquete

- **Bloques fijos**

- Cada dato a enviar posee una longitud fija
- Fácil de serializar
- Subóptimo con tipos de longitud variable (e.g. *strings*)

- **Bloques dinámicos**

- Separador para delimitar comienzo y terminación de un campo
- Longitud del tipo para delimitar longitud del campo
- Pequeño overhead al agregar *bytes* extra

- **Esquema mixto**

- Parámetros fijos (e.g. *integers*) no llevan delimitadores / longitud
- Parámetros variables llevan delimitadores / longitud



Mensajes | Longitud de Paquete (Ejemplo: TLV)

- Todos los parámetros siguen el formato Type - Length - Value
 - **Type:** Indica el tipo de dato / entidad. Tamaño fijo
 - **Length:** Longitud del value sin contar el tipo y el length. Tamaño fijo
 - **Value:** Dato a enviar. Tamaño variable. **Admite subtipos**
- Ejemplo: Encodear una dirección: Alsina 2B

Mini protocolo

```
Tipo 01: Integer (4 bytes)
Tipo 02: string (variable)
Tipo 03: Dirección (compuesto)
Tipo 03.01: Calle (string)
Tipo 03.02: Número (integer)
Tipo 03.03: Unidad (string)
```

Serialización

```
03 11 01 06 41 6c 73 69 6e 61 02 04 00 00
00 02 03 01 42
```

Serialización (desglose)

```
03 11 (Dirección)
    01 06 41 6c 73 69 6e 61 (Calle)
    02 04 00 00 00 02 (Número)
    03 01 42 (Unidad)
```



Mensajes | Longitud de Paquete (Ejemplo: TFTP)

Read/Write Packet

Opcode = 1/2	Filename	0	Mode	0
2 bytes	string	1 byte	string	1 byte

Data Packet

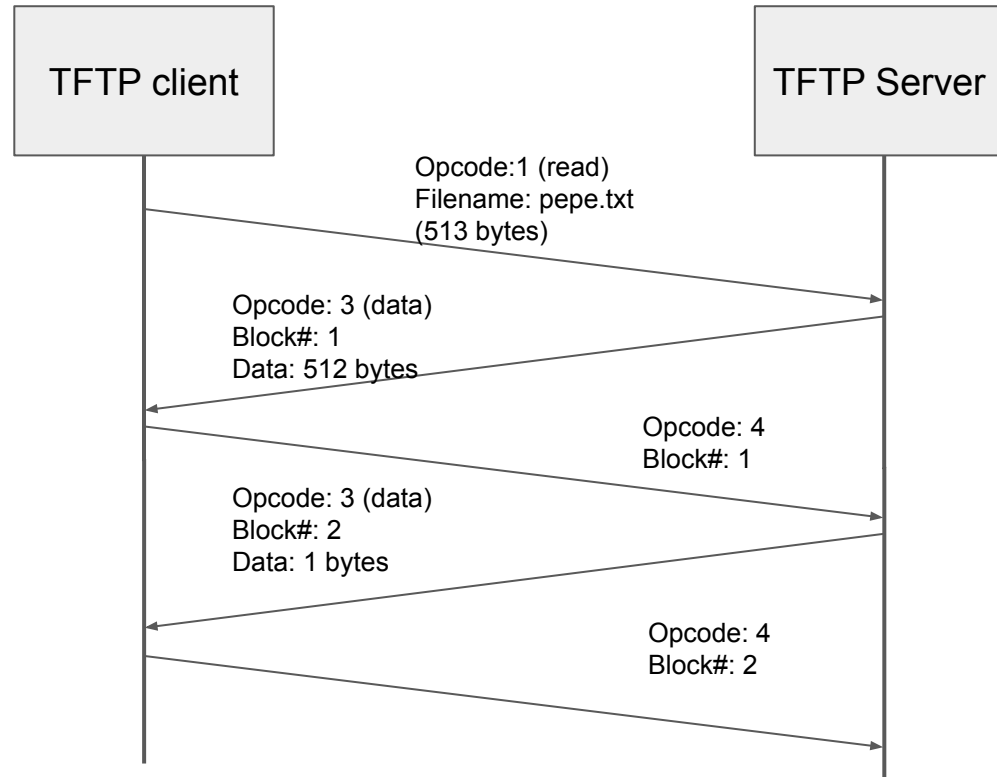
Opcode = 3	Block #	Data
2 bytes	2 bytes	N bytes

ACK Packet

Opcode = 4	Block #
2 bytes	2 bytes

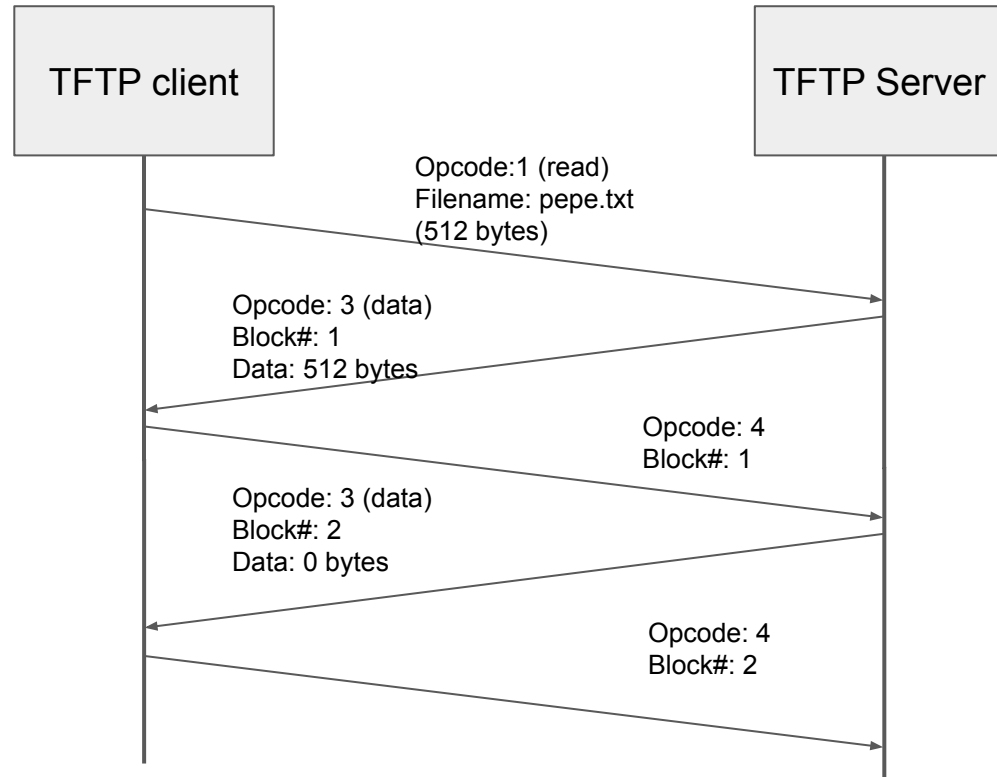


Mensajes | Longitud de Paquete (Ejemplo: TFTP)





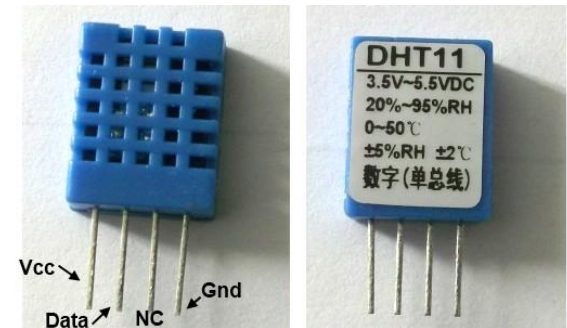
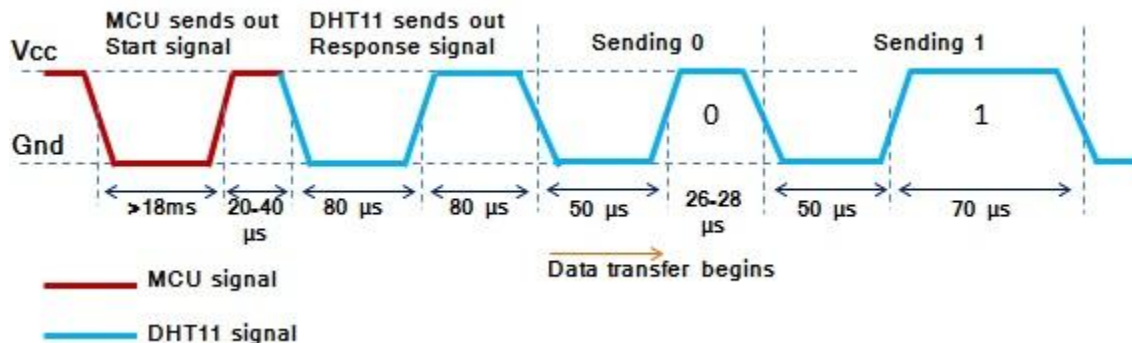
Mensajes | Longitud de Paquete (Ejemplo: TFTP)





Mensajes | Sincronización de señales

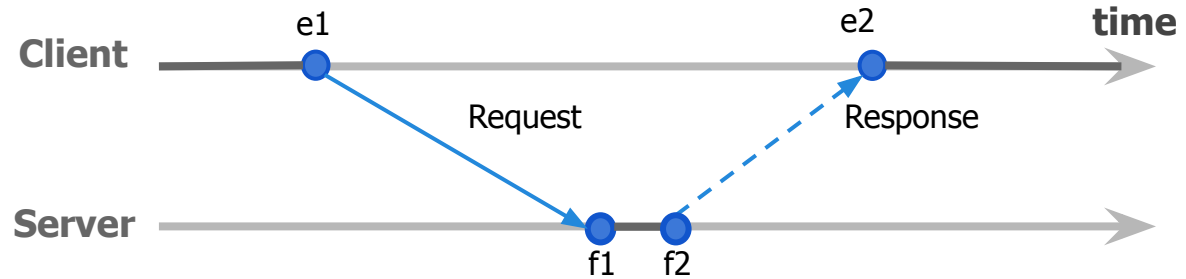
- En ciertos protocolos es necesario saber dónde empieza una señal para entender la secuencia de paquetes.
- **Ejemplo bajo nivel: DHT11 (sensor de humedad y temperatura)**
 - DHT11 envía data sobre pin de comunicación
 - Comienzo con una señal de sincronismo
 - Data (40-bit) = Integer Byte of RH + Decimal Byte of RH + Integer Byte of Temp. + Decimal Byte of Temp. + Checksum Byte



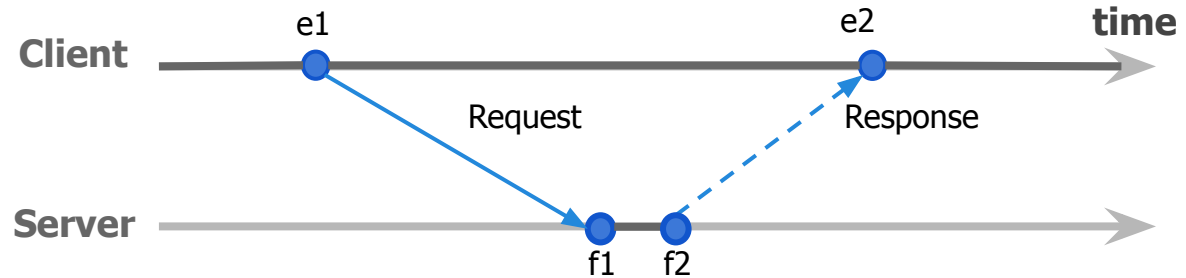


Mensajes | Sincrónicos vs Asincrónicos

Sincrónico



Asincrónico

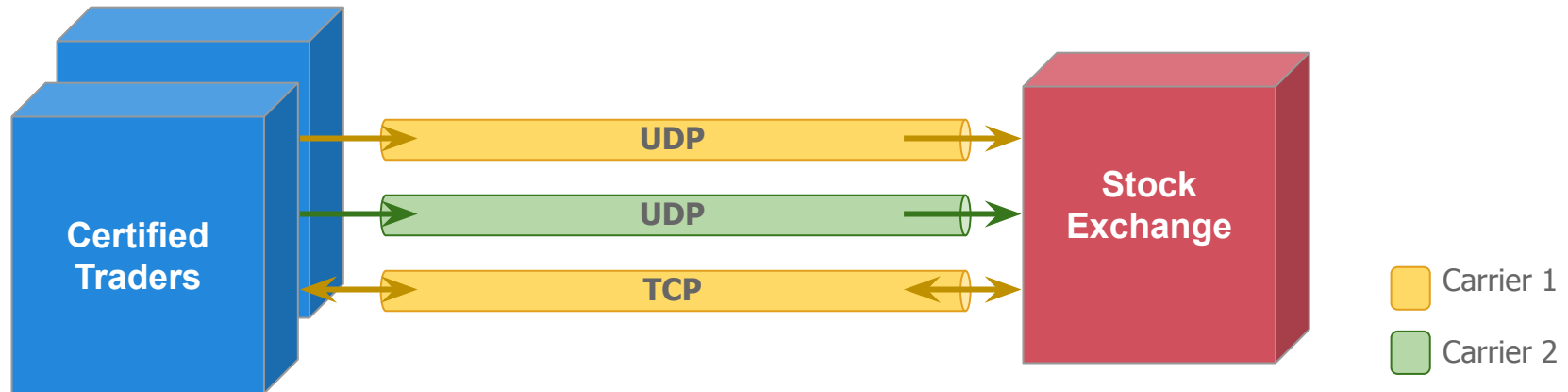




Protocolos | Ejemplo Múltiples Canales

Caso de Uso:

- Bolsa de comercio recibiendo información de muchos Operadores distribuidos
- Tráfico de paquetes con picos de transferencia
- Canales UDP para garantizar throughput (aún con repetidos)
- Canal TCP para pedir retransmisión



Agenda



- ☐ Modelos de Capas: Layers y Tiers
- ☐ Interfaces
- ☐ Protocolos
- ☐ Mensajes
- ☒ **RESTful**



Representational State Transfer (aka REST/RESTful)

- Basado en entidades (*Web Resources*)
- Cada Web resource es representado por una **URI**
- **HTTP/HTTPS** utilizado como protocolo de comunicación
- **JSON/XML** utilizado como protocolo de serialización
- Cambio de estados a través de operaciones **CRUD**

{ REST }

Operación	Verbo HTTP	Equivalente SQL
Create	POST	INSERT
Read	GET	SELECT
Update	PUT	UPDATE
Delete	DELETE	DELETE

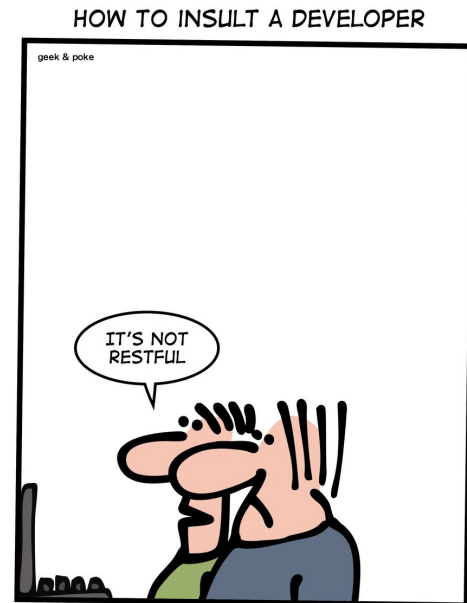


Objetivos:

- Alta performance, escalabilidad, confiabilidad, etc.

Principios de Arquitectura:

- Arquitectura cliente/servidor
- *Cacheability*
- Interface Uniforme:
 - Hypermedia As The Engine Of Application State (HATEOAS)
- *Statelessness*
- *Layered system*





RESTful | Ejemplo | httpbin.org + Flasagger

- <https://httpbin.org/>:
 - Servicios de ejemplos RESTful online
- <https://github.com/flasgger/flasgger>
 - Port de Swagger para Python.
 - Genera interfaz de exploración y documentación de APIs.
 - Librería de documentación RESTful APIs utilizada por httpbin.org.

httpbin.org ^{0.9.2}

DELETE	/delete The request's DELETE parameters.
GET	/get The request's query parameters.
PATCH	/patch The request's PATCH parameters.
POST	/post The request's POST parameters.
PUT	/put The request's PUT parameters.

Auth Auth methods





RESTful | Ejemplo | GitLab

- Plataforma *DevOps* incluyendo manejo de proyectos, control de versiones, *builds* y *deployments*.
- Modelo mixto de interfaz
 - Orientada a Entidades. Ej.:
 - /projects
 - /groups
 - /users
 - /snippets
 - /runners
 - Orientada a Procesos o Acciones. Ej.:
 - /search
 - /lint



RESTful | Ejemplo | GitLab | Obtener Proyectos

```
$ curl --verbose -H Accept:application/json -X GET https://gitlab.com/api/v4/projects
> GET /api/v4/projects HTTP/1.1
> Host: gitlab.com
> Accept:application/json
>
...
< HTTP/1.1 200 OK
< Content-Type: application/json
< Content-Length: 2918
...
< [{ "id": 3472737,
    "description": "Inkscape vector image editor",
    "name": "inkscape",
    "web_url": "https://gitlab.com/inkscape/inkscape",
    "star_count": 1726, ...
    "namespace": { "id": 470642, "name": "Inkscape", "web_url": "https://gitlab.com/
groups/inkscape", ... }
  }, ... ]
```



RESTful | Ejemplo | GitLab | Obtener Referencias Anidadas

```
$ curl -X GET
https://gitlab.com/api/v4/projects/3472737/
users
...
< [ {
  "id": 43416, "name": "Felipe Sanches",
  "username": "fsanches",
  "state": "active", "avatar_url":
"https://secure.gravatar.com/avatar/3374412
c23252294142453a60b066300?s=80&d=identicon"
,
  "web_url":
"https://gitlab.com/fsanches"
},
...
]
```

```
$ curl -X GET
https://gitlab.com/api/v4/users/43416
...
< { "id": 43416, "name": "Felipe Sanches",
  "username": "fsanches",
  "state": "active", "avatar_url":
"https://secure.gravatar.com/avatar/3374412
c23252294142453a60b066300?s=80&d=identicon"
,
  "web_url": "https://gitlab.com/fsanches",
  "created_at": "2014-07-21T19:32:00.713Z",
  "bio": null, "location": null,
  "public_email": "", "skype": "",
  "linkedin": "", "twitter": "",
  "website_url": "",
  "organization": null, "job_title": "",
  "work_information": null
}
```



RESTful | Ejemplo | GitLab | Crear Tags

```
$ curl -H Content-Type:application/json -X POST /projects/3472737/repository/tags  
--data '{"tag_name":"stable", "ref":"master"}'
```

```
{  
  "commit": {  
    "id": "2695effb5807a22ff3d138d593fd856244e155e7",  
    "short_id": "2695effb",  
    "title": "Initial commit",  
    "created_at": "2017-07-26T11:08:53.000+02:00",  
    "parent_ids": [  
      "2a4b78934375d7f53875269ffd4f45fd83a84ebe"  
    ],  
    "message": "Initial commit",  
    "author_name": "John Smith",  
  }, ...  
  "name": "stable",  
  "target": "2695effb5807a22ff3d138d593fd856244e155e7",  
  ...  
}
```



```
$ curl --verbose -X DELETE  
https://gitlab.com/api/v4/projects/3472737/repository/branches/issue-123  
...  
< HTTP/1.1 204 No Content
```



RESTful | Ejemplo | GitLab | Status Codes

200 OK	The GET, PUT or DELETE request was successful, the resource itself is returned as JSON.
204 No Content	The server has successfully fulfilled the request and that there is no additional content to send in the response payload body.
201 Created	The POST request was successful and the resource is returned as JSON.
304 Not Modified	Indicates that the resource has not been modified since the last request.
400 Bad Request	A required attribute of the API request is missing, e.g., the title of an issue is not given.
401 Unauthorized	The user is not authenticated, a valid user token is necessary.
403 Forbidden	The request is not allowed, e.g., the user is not allowed to delete a project.
404 Not Found	A resource could not be accessed, e.g., an ID for a resource could not be found.
405 Not Allowed	The request is not supported.
409 Conflict	A conflicting resource already exists, e.g. creating a project with a name that already exists
412	Indicates the request was denied. May happen if the If-Unmodified-Since header is provided when trying to delete a resource, which was modified in between.
422 Unprocessable	The entity could not be processed.
500 Server Error	While handling the request something went wrong server-side.



RESTful | Identidad

Identificar una unívocamente una entidad entre diferentes sistemas es una prioridad

- Identificador debería tener información sobre la identidad que referencia

Identidad != Nombre

- Nombre puede cambiar, identidad no (Ej. <https://tracker.com/pet?name=Lassie>)
- URL define la identidad de la entidad, no el identificador (Ej. <https://tracker.com/pet/12345>)

```
// Pet entity. Bad ID
{
  "id": "12345",
  "type": "/dog",
  "name": "Lassie"
}
```

```
// Pet entity. Better ID
{
  "id": "/pet/12345",
  "type": "/dog",
  "name": "Lassie"
}
```



- Identificar correctamente una entidad ayuda a la integración de sistemas
- **Integración con otros sistemas**
 - Siempre usar URIs para identificar entidades
 - URIs pueden ser relativas si la entidad es del mismo sistema

```
// Bad relationship
{
  "id": "12345",
  "type": "/dog",
  "name": "Lassie",
  "owner": "98765",
}
```

```
// Better relationship
{
  "id": "/pet/12345",
  "type": "/dog",
  "name": "Lassie",
  "owner": "/users/98765",
}
```



Semantic Versioning (semver):

- Estándar más utilizado para definir versiones de APIs y Librerías.
- Foco en brindar información de retrocompatibilidad de la interfaz.
- Ej.:

1.7.1

Major . Minor . Patch

- **Incremento de números de versión:**
 - **Major:** al introducir cambios incompatibles con la versión anterior.
 - **Minor:** al agregar funcionalidad pero mantener retrocompatibilidad
 - **Patch (aka build):** al introducir correcciones que no afectan la interfaz.



- **Tipos**

- **Versionado explícito en URL**

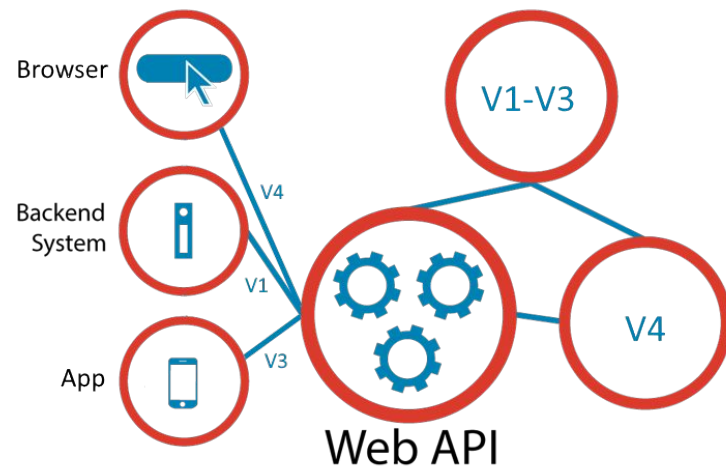
- Fácil de usar y testear
 - RESTful compliant (Interface uniforme)

- **Versionado via HTTP Custom header**

- Semánticamente incorrecto.

- **Versionado via HTTP Accept header**

- Semánticamente correcto. Indica cómo se quiere obtener el recurso
 - Díficil de testear





RESTful | Versionado de Objetos

- **Versionado de objetos != Versionado de API**
- Tipos de versiones de objetos:
 - *Format Versioning*: la API pueden brindar distintas representaciones de **la misma entidad** y así cambiar su formato.
 - Ej.1 : Invocaciones desde distintas versiones de una misma API
 - Ej.2 : Al filtrar de datos del objeto resultante
 - *Historical Versioning*: la misma entidad tiene distintas versiones que fueron almacenadas a lo largo de su ciclo de vida.
 - Ej. : API de páginas en una Wiki

```
GET /api/pages/{id}/{rev}
GET /api/pages/{id}
GET /api/pages/{id}/latest
GET /api/pages/{id}?version={rev}
```



- Standard de facto de la industria
- Basado en entidades y cambios de estado
 - Ideal manejo de recursos
 - Complejo para ejecución remota de lógica de negocio
- La teoría es importante. También lo es ser pragmático: hay que encontrar un equilibrio
- ***Don't mess it up with the basics !***
 - Utilizar HTTP/JSON directamente
 - Identificar unívocamente entidades con URLs (no con nombres)
 - Manejar correctamente versionado y queries





Bibliografía

- M. Van Steen, A. Tanenbaum: Distributed Systems. 3rd Edition. Pearson Education, 2017.
 - Capítulo 2.1 - Architectures, Architectural Styles
 - Capítulo 4 - Communication
- Microsoft Application Architecture Guide, 2nd Edition. 2009.
 - Capítulo 5: Layered application guidelines
 - Capítulo 19: Physical tiers and deployment
- Martin Nally. Designing Quality APIs. 2018.
 - <https://www.youtube.com/watch?v=P0a7PwRNLVU>